This logic breaks whenever Carol switches teams or someone else replaces her. A more stable approach is to model the management relationship itself:

```
when { principal in ManagerOf::"team:finance".members };
```

Representing “manager of team finance” through a relationship graph captures intent without relying on specific names.

At ACME, these experiences emphasized a key lesson: authorization logic is only as durable as the data it relies on. By treating roles, teams, and relationships as governed attributes rather than *ad hoc* fields, ACME’s identity team ensured that policy stability is supported by data stewardship.

10.1.4 Granularity and grouping

Policies are most effective when they operate at the right level of detail. Fine-grained actions improve policy accuracy by differentiating activities such as viewing, editing, or sharing. However, exposing every fine-grained action directly to employees or customers can be overwhelming and hard to manage. To keep things practical, organizations often group related actions into broader roles or bundles that are understandable to end users; for example, offering a single collaborator role in the UI that corresponds to several underlying capabilities.

ACME reps discovered that showing detailed actions directly to customers and employees on the Customer Collaboration platform caused confusion, so they grouped those actions into broader roles in the UI, such as collaborator or viewer. However, behind the scenes, the platform’s early policies used a single catch-all action, `doc:all`, which combined all capabilities. This made writing policies easier, but it also erased important differences; there was no way to determine if someone should be allowed to view, edit, or share a document.